

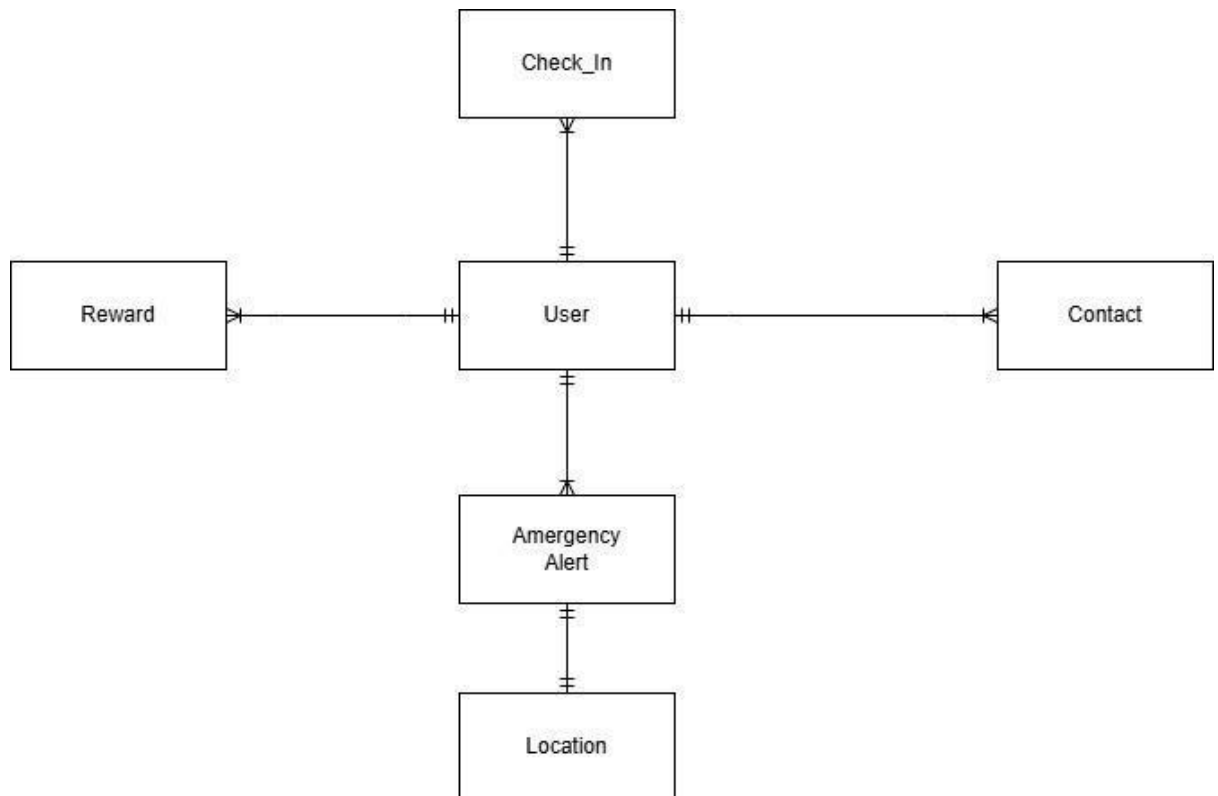


Figure 4.3: ERD diagram that describes the relationship between the entities

4.3.1 Entities attributes

Core Entities and Attributes

User/Profile - authenticated UUID, name, contact and address data, blood type, inactivity threshold, escalation preference, phone verification, Terms acceptance, Legacy UID and Legacy consent/testing controls.

Check-In - owner UUID, timestamp and status used to calculate the rolling threshold, history and streaks.

Inactivity Monitor Status - per-user threshold snapshot, latest check-in, missed-window count, last notification stage, SMS attempt state and reset timestamps.

Contact - owner UUID, name, relationship, phone, address, primary flag and phone verification timestamp. Database logic preserves at most one primary contact.

Emergency Alert and Location - owner UUID, trigger source, status, timestamp and optional coordinates or Maps link.

Reward Catalogue and Earned Reward - virtual badge/voucher definition, milestone, active state, owner claim, claim timestamp and stable voucher code. Admin policies protect catalogue mutation.

Legacy Preference, Note and Document - owner-scoped funeral selections, protected notes, private object paths and upload timestamps.

Legacy Release and Access - owner warning, 24-hour grace deadline, cancellation token state, seven-day contact window, OTP verification and access audit events.

OTP and Delivery Outbox - hashed expiring verification codes, attempt counters, consumption timestamps, idempotency keys, provider responses, retries and final delivery state.

Main Relationships

A user owns many check-ins, contacts, emergency alerts, rewards, notes and documents. Each contact, check-in and protected item belongs to one user. One user has one current inactivity-monitor row and can designate one primary contact. Legacy releases and delivery records reference the owner and preserve auditable state transitions.

4.4 Security Handling

4.4.1 Authentication and Access Control (Supabase Authentication)

The system uses **Supabase Authentication** to manage user login and identity verification. Users are required to log in using their registered credentials, and authentication is handled using secure token-based sessions (JWT). This ensures that only authenticated users can access system features such as check-ins, emergency alerts, and personal documents. Unauthorized users are restricted from accessing protected resources.

4.4.2 Data Protection (Supabase Database & Storage)

All user data, including personal details, contacts, check-in records, and documents, are stored securely in the **Supabase database and storage services**. Data access is controlled using Supabase policies, ensuring that users can only access their own data. Sensitive files such as digital wills are stored securely using controlled storage access, preventing unauthorized viewing or modification.

4.4.3 Secure Communication (HTTPS Protocol)

The Flutter mobile application communicates with Supabase and external APIs using **HTTPS protocols**. This ensures that all data transmitted, including login credentials and location data, is encrypted during transmission. As a result, the system protects against data interception and man-in-the-middle attacks.

4.5 UI Prototype

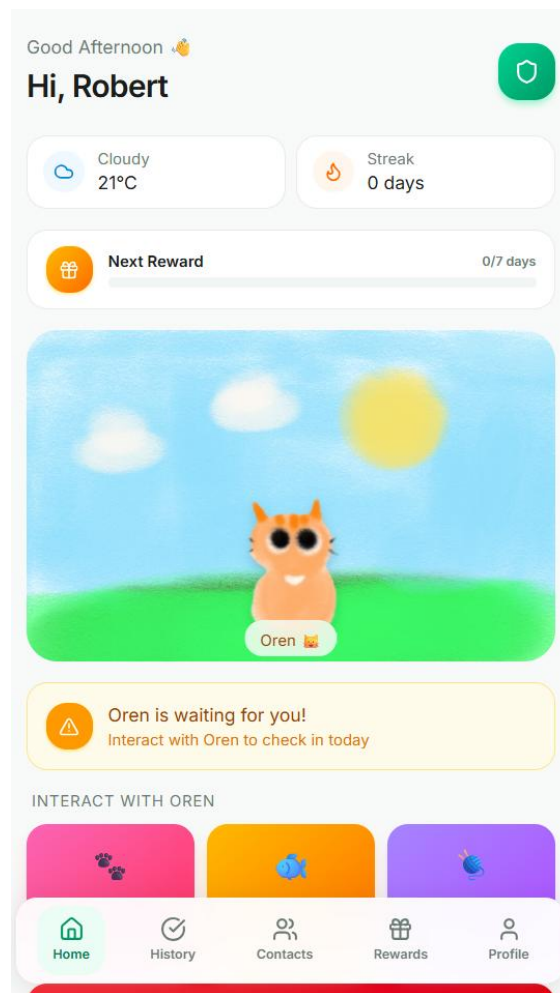


Figure 4.4.1: Home Page

The Home page places Oren and the weather-responsive pixel scene near the top. Borderless status text clearly separates Oren's mood from actions. Feed and Play appear in one row, owned items can be selected, and the token bar and Shop control show purchasing capacity. The safety check-in control is visually distinct from care actions so users do not mistake feeding or playing for a heartbeat. The Safety Monitor appears lower on the page with threshold and reminder-stage information.

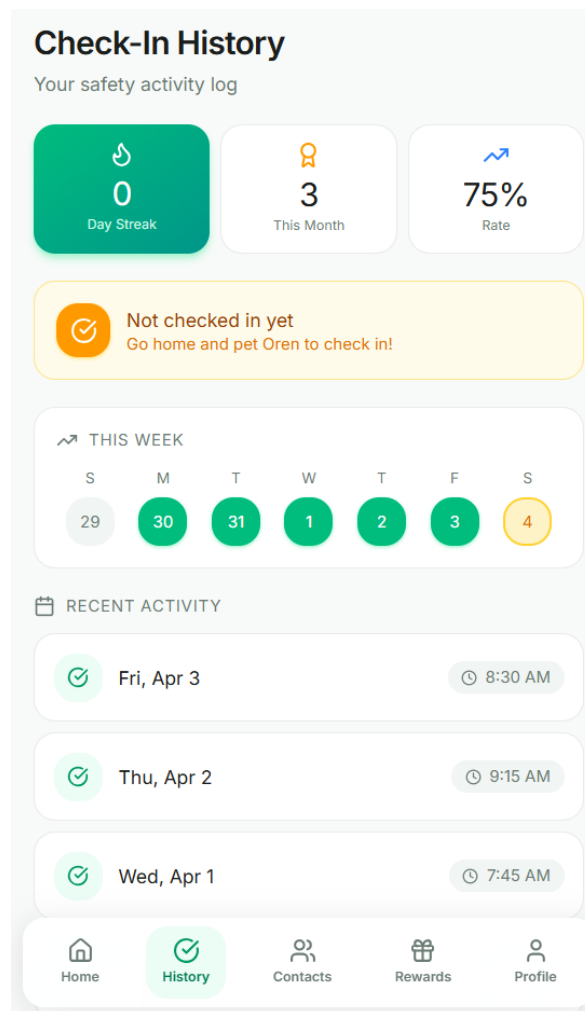


Figure 4.4.2: Check-In History Page

The Check-In History page lists successful heartbeat timestamps and shows whether the current rolling threshold is still valid. It avoids an ambiguous percentage rate and instead explains the latest check-in and due state using the user's selected threshold.

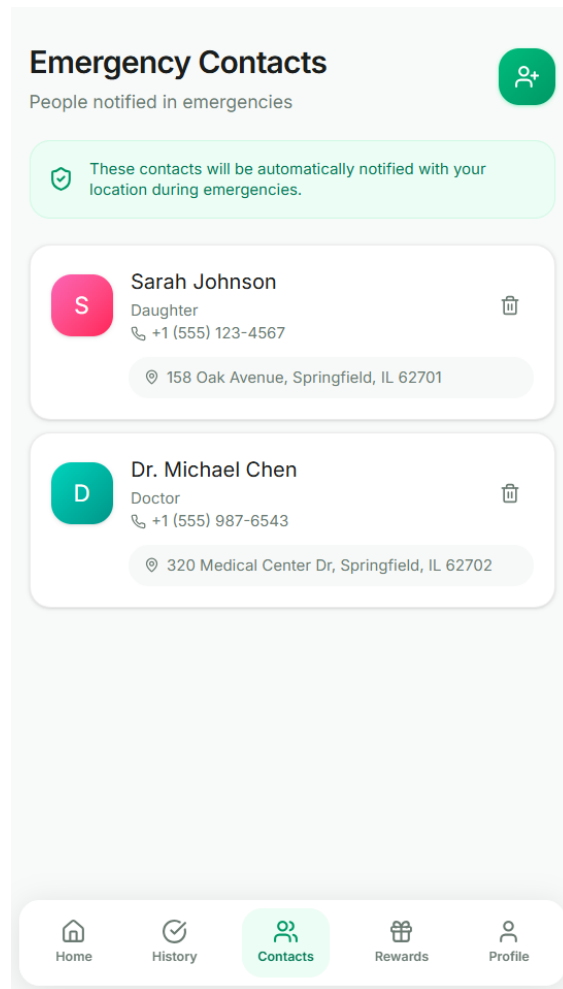


Figure 4.4.3: Emergency Contacts Page

The Emergency Contacts page presents up to five owner-managed contacts with a clear primary-contact indicator, call action and verified phone details. Oren's phone-call pose and concise guidance explain that the selected primary contact receives eligible SOS and inactivity alerts. Add, edit, delete and primary-selection actions remain visually separate.

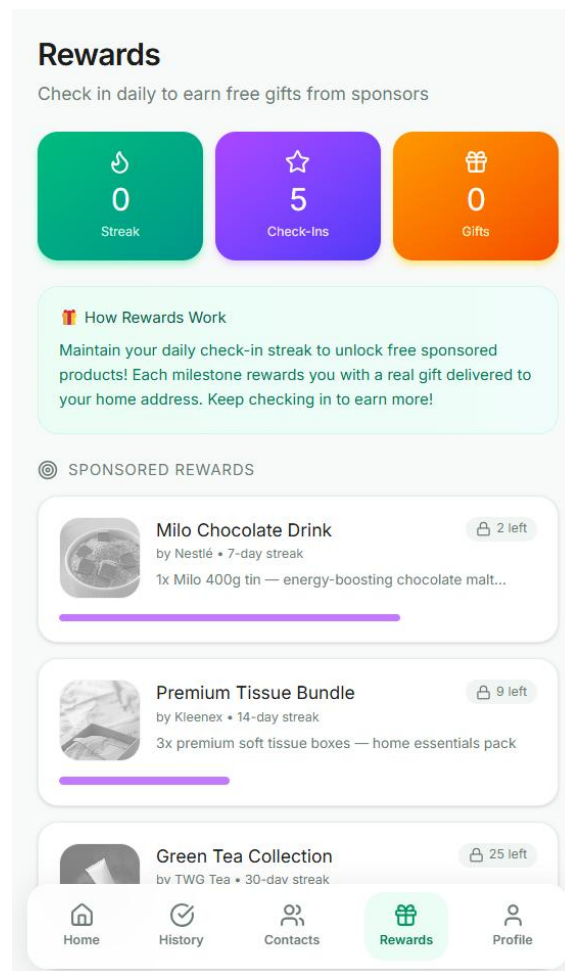


Figure 4.4.4: Rewards Page

The Rewards page contains virtual badges and vouchers rather than physical delivery items. Eligible badges can be collected into My Badge List, while Check Reward opens details for badges or vouchers and displays a stable user-specific redemption code when applicable. A separate authorised admin route maintains catalogue entries with Realtime updates and manual refresh fallback.

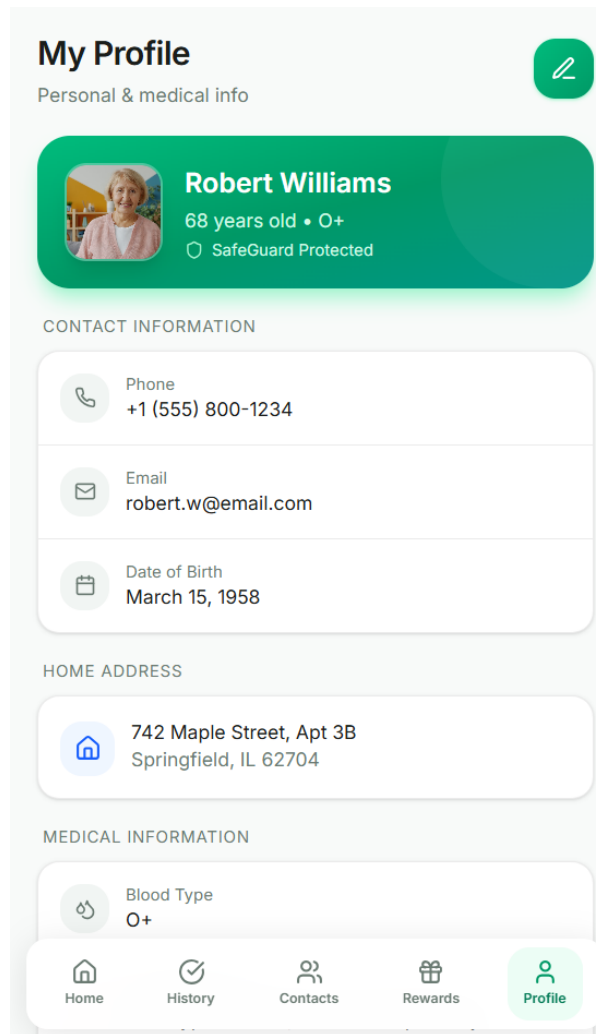


Figure 4.4.5: Profile Page

The Profile page displays and edits the user's contact, address, blood type, rolling inactivity threshold and escalation preference. It shows phone verification state, provides a copyable Legacy UID, allows biometric session protection on supported devices, and links to the Terms and Conditions and Legacy Planning. The page also reopens the feature guide and provides Settings for local reminders, Oren sounds, text size, reduced motion, biometric controls and restore-defaults. Age and a public profile photograph are not required by the current data model.

4.6 Chapter Summary and Evaluation

This chapter presented the current layered client/cloud architecture, activity flows, data entities, security controls and user-interface design. The design separates local usability state from cloud authority, uses scheduled workers for safety and Legacy timing, and protects owner data through authenticated repositories, Row Level Security, private Storage and short-lived access. The preserved diagrams should be read together with the updated textual rules where the implementation evolved during Agile development.

Chapter 5

Implementation and Testing

5 Implementation and Testing

This chapter explains how Ethernacare was translated from the requirements and system design into a working safety, well-being and legacy-planning application. It covers the system architecture, implemented modules, representative screens, test strategy, context-based test cases and current evaluation evidence.

Development was iterative and evidence-driven. Business rules were isolated in services and repositories for deterministic testing, while high-risk workflows use persisted server state, database controls and auditable provider boundaries. Interface changes were checked across compact mobile and wider desktop layouts.

5.1 System Implementation

Ethernacare uses a layered Flutter-and-Supabase architecture. Flutter widgets form the presentation layer, controllers and services apply reusable business rules, and repositories isolate Supabase Auth, PostgreSQL, Storage, RPC and Edge Function calls. Android WorkManager provides best-effort local reminder evaluation. Independently, Supabase Cron invokes the inactivity-threshold worker every 15 minutes and the Legacy worker daily at 12:00 a.m. Malaysia time, so authoritative server processing does not depend on the application remaining open.

The application separates local continuity from cloud authority. SharedPreferences stores user-scoped Oren care, weather, dashboard, reward, chat, reminder and first-run-guide state for responsive reopening on the same device; device preferences store reminder, sound, text-size and reduced-motion choices. Supabase remains the authority for authentication, profiles, contacts, check-ins, emergency alerts, reward catalogues, earned rewards, Legacy Planning records, private document metadata, OTPs, delivery outboxes and seven-day Legacy access windows.

5.1.1 Development Tools and Technologies

Technology	Use in Ethernacare	Reason for selection
Flutter 3.38.5 / Dart 3.10.4	Responsive Android, Windows, and web client	One typed codebase with reusable Material widgets and mature test support

Technology	Use in Ethernacare	Reason for selection
Supabase Auth	Email/password, 8-digit email confirmation, password recovery, and configured OAuth	Managed sessions integrated with PostgreSQL user identities
Supabase PostgreSQL	Profiles, contacts, check-ins, rewards, emergency records, OTPs, and Legacy access state	Relational integrity, constraints, triggers, RPCs, and auditable SQL migrations
Row Level Security	Restricts authenticated rows to their owner and protects admin-only reward operations	Database-enforced authorization even when a client request is modified
Supabase Storage	Private PDF/JPEG/PNG Legacy documents up to 10 MB	Private buckets, owner folders, and short-lived signed URLs
Deno / TypeScript Edge Functions	SMS OTP, emergency SMS, AI guidance, threshold processing, Legacy access/cancellation and daily Legacy processing	Server-side secrets and narrow privileged operations outside the Flutter client
Supabase Cron / pg_cron	Runs the threshold worker every 15 minutes and the Legacy worker daily at midnight MYT	Cloud execution continues when the phone is closed or offline
Python FastAPI reference worker (non-production)	Readable reference implementation in server/python_legacy_server	Documents the 90-day domain workflow; Supabase Edge Functions are the deployed production workers
Twilio and Brevo	SMS delivery and transactional email	Provider APIs with server-side credentials and delivery logs

Technology	Use in Ethernacare	Reason for selection
WorkManager and local notifications	Best-effort Android local reminders and notification presentation	Improves local responsiveness without acting as the authoritative SMS scheduler
data.gov.my and Open-Meteo	Malaysia regional weather with coordinate fallback	Official source first, then a resilient public fallback
Flutter Test / lints / coverage	Logic, service-boundary, widget, responsive, and regression verification	Repeatable evidence before release builds

5.1.2 Modules and Code Snippets

5.1.3 Sample Screens

The supplied guide asks for a sample screen for each module or sub-module. Figure 5.1 is a verified mobile-web capture generated from the current build. Table 5.2 defines the remaining current captures that should be inserted when assembling the final full report. Older prototype screenshots should not be reused as current evidence because the Home, Contacts, Rewards, Profile, and Legacy layouts changed during implementation.

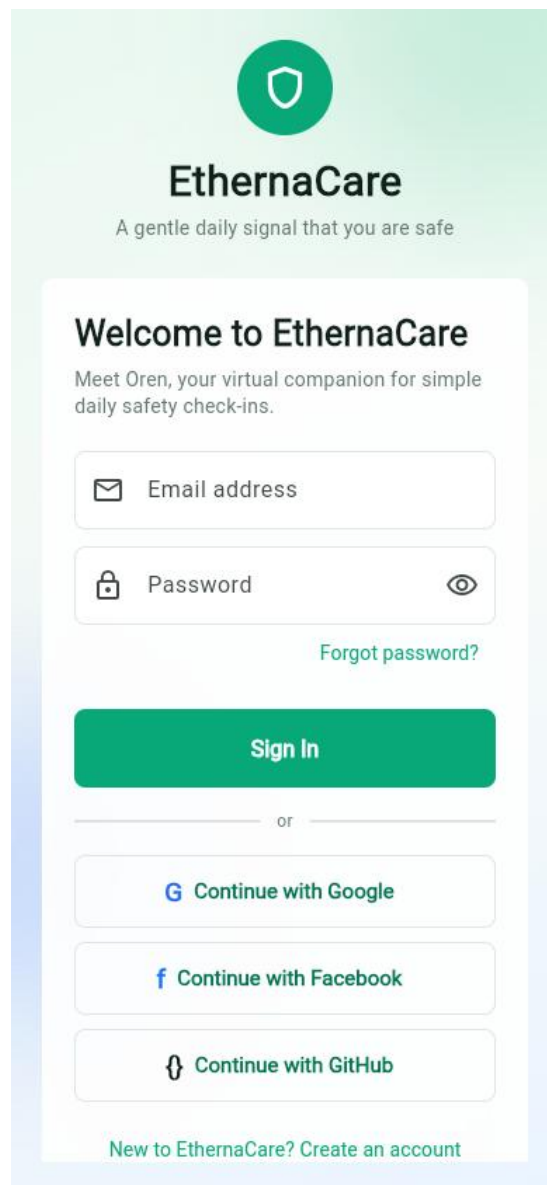


Figure 5.1.1: Current Ethernacare authentication screen at 390 x 844 pixels

Figure	Current screen to capture	Evidence to show
5.2	Home - Oren and weather	Oren status above the scene, token/shop controls, Feed and Play row, selected toy, and Safety Monitor lower on the page
5.3	Check-in History	Threshold-aware current status and timestamp list without the removed check-in-rate percentage

Figure	Current screen to capture	Evidence to show
5.4	Trusted Contacts	Primary-contact indicator, call action, verified phone, and Oren phone-call pose
5.5	Virtual Rewards	Badge and voucher catalogue, Check Reward action, My Badge List, and voucher redeem code
5.6	Profile	Editable threshold, escalation target, phone verification state, copyable Legacy UID, biometrics, feature-guide replay, Settings, and Terms link
5.7	Legacy Planning	Funeral preference selections, authorized existing contact, safe notes warning, secure documents, and owner enable switch
5.8	Legacy Check	UID and primary phone verification, SMS code, preferences access, protected-content status, days remaining or seven-day expiry
5.9	Reward Admin catalogue	Separate admin authentication, create/edit controls, Realtime/manual refresh, and eligible delete action

Table 5.1.2: Required current sample-screen set

5.2 Testing Methods

5.2.1 Functional Testing

Unit and logic testing validates threshold boundaries, escalation rules, Oren energy decay, reward calculations, validation, serialization, Legacy content filtering, and provider fallback decisions.

- Service-boundary testing uses mocked repositories and Supabase responses to verify orchestration without changing production records.
- Widget testing renders authentication, Oren, contacts, rewards, Legacy, dialogs, and guidance states at controlled viewports to detect unreachable controls and overflow.
- Integration checks verify deployed Edge Function health, migration parity, private-storage expectations, release builds, and platform manifests.
- Context-based testing describes the real situation of a user, owner, primary contact, or reward administrator instead of testing an isolated button without purpose.

5.2.2 Non-Functional Testing

Reliability testing checks duplicate suppression, one-time escalation, retry visibility, stale-processing recovery, provider timeouts, cache restoration, app-resume refresh, overlapping dashboard requests, serialized Oren actions and fallback behavior. Security testing checks RLS ownership, recent phone verification, private Storage paths, signed URLs, OTP expiry and attempts, Legacy audit events, admin authorization, verified SMS recipients, approved message categories and server-secret boundaries. Usability checks cover scrolling, safe areas, readable status text, semantic labels, guidance, Settings, large text, reduced motion, full-page forms and mobile viewport behavior. Maintainability is supported by repositories, services, typed models, idempotent migrations and separate Edge Functions.

The current automated suite contains 138 passing tests. Line coverage is 2,515 of 8,208 lines, or 30.64 percent. The suite covers important domain and widget behaviour, including biometric prompt coordination, rolling inactivity logic, first-run guide persistence, device settings and concurrent Oren care actions. Line coverage remains modest for the full presentation and provider-integration surface, so it is reported as a limitation rather than complete assurance. Real carrier delivery, Android Doze/background execution, biometric hardware and GPS accuracy still require physical-device acceptance testing.